

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 5. Container Orchestration

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate > {
```

```
public static void main(String[] args)

</servlet>
<servlet-name>BoardController</servlet-name>
<servlet-class>com.joneconsulting.controller.BoardCont
<init-param>
  <param-name>user_name</param-name>
  <param-value>henneth Lee</param-value>
</init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명

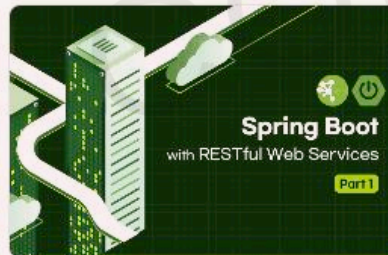


Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- **Section 5: Container Orchestration**
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 5.

Container Orchestration

- 컨테이너 오케스트레이션 도구
- Docker Swarm을 이용한 컨테이너 관리
- Rolling Updates & Rollback

Orchestration Tools란?

- 컨테이너의 배포, 관리, 확장, 네트워킹을 자동화 해주는 도구

| <https://landscape.cncf.io/> → Orchestration & Management

- 수백, 수천 개의 컨테이너와 호스트를 배포하고 스케줄링하기 위해 사용되는 도구

- 컨테이너 오케스트레이션

| 프로비저닝 및 배포

| 구성 및 일정 조정

| 리소스 할당

| 컨테이너 가용성

| 컨테이너 스케일링

| 로드밸런싱 및 트래픽 라우팅

| 컨테이너 상태 모니터링

Docker Swarm

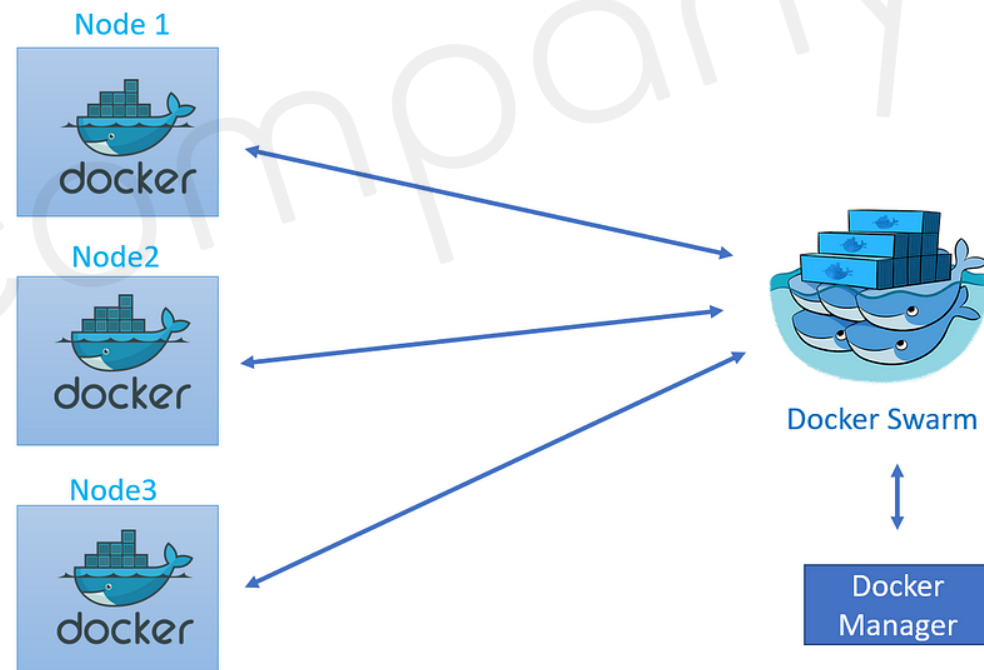
- 여러 docker host를 클러스터로 묶어주는 컨테이너 오케스트레이션

이름	역할	대응하는 명령어
Compose	여러 컨테이너로 구성된 Docker 애플리케이션을 관리 (주로 단일 호스트)	docker-compose
Swarm	클러스터 구축 및 관리 (주로 멀티 호스트)	docker swarm
Service	Swarm에서 클러스터 안의 서비스 (컨테이너 하나 이상의 집합)을 관리	docker service
Stack	Swarm에서 여러 개의 서비스를 합한 전체 애플리케이션을 관리	docker stack

Docker Swarm을 이용한 컨테이너 관리 ①

멀티 노드의 Swarm Cluster 구성하기

- Manager 노드에 Swarm 초기화 설정
 - | Manager 노드에 docker swarm init 설정 → Swarm 모드 활성화
 - | Worker 노드를 Manager 노드에 등록 → Token 필요



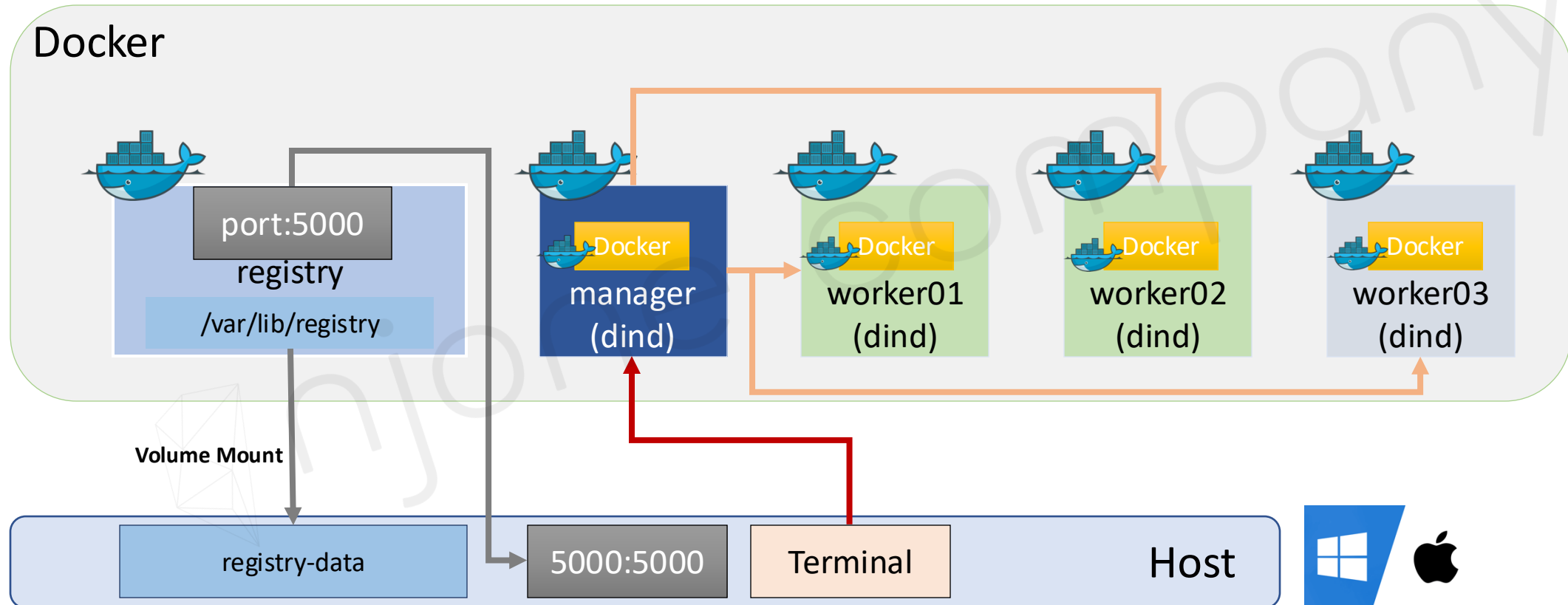
Docker Swarm Architecture

Docker Swarm을 이용한 컨테이너 관리 ①

Docker Swarm을 이용한 시스템 구축

- Docker in Docker (dind)

도커 컨테이너 안에서 도커 호스트 실행



Docker Swarm을 이용한 컨테이너 관리 ①

Docker Swarm을 이용한 시스템 구축

- Manager에 Swarm 초기화 설정 (docker exec 명령어 사용)

- | Manager Container에 Swarm 모드 활성화 → docker swarm init

- | Worker Container를 Manager Container에 등록 → docker swarm join

```
$ docker exec -it manager docker swarm init
```

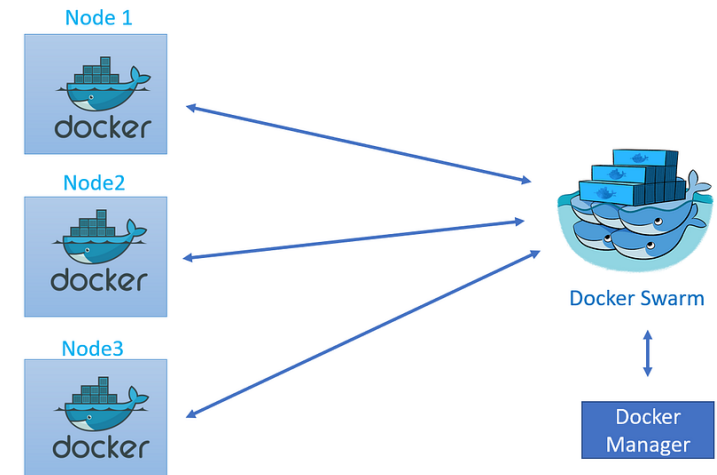
```
$ docker exec -it worker01 docker swarm join --token [JOIN TOKEN] manager:2377
```

- Swarm에서 사용할 포트

- | TCP port 2377: cluster management 통신에 사용

- | TCP/UDP port 7946: node간의 통신에 사용

- | TCP/UDP port 4789: overlay network 트래픽에 사용



Docker Swarm을 이용한 컨테이너 관리 ①

Docker Swarm을 이용한 시스템 구축

- Docker Registry용 이미지 생성

| 이미지명 → <registry_hostname>/<repository_name:tag_name>

```
$ docker tag example/echo:latest localhost:5000/example/echo:latest
```

- Docker Registry용 이미지 등록

```
$ docker push localhost:5000/example/echo:latest
```

- Worker Container에 컨테이너 생성 (worker01 실행)

```
# docker pull registry:5000/example/echo:latest
```

```
# docker image ls
```

Docker Swarm을 이용한 컨테이너 관리 ②

Docker Swarm Service 활용하기

- Swarm Service (manager 실행)

| 애플리케이션을 구성하는 일부 컨테이너(단일 또는 복수)를 제어하기 위한 단위

```
# docker service create --replicas 1 --publish 80:80 --name my-nginx nginx:latest
```

```
# docker service ls
```

```
# docker service scale my-nginx=3
```

```
# docker service ps my-nginx
```

```
# docker service rm my-nginx
```

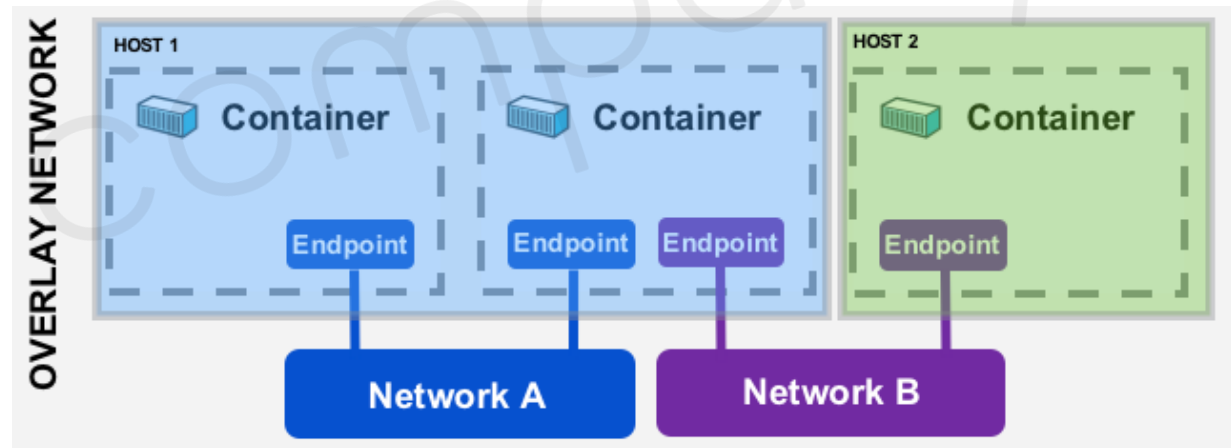
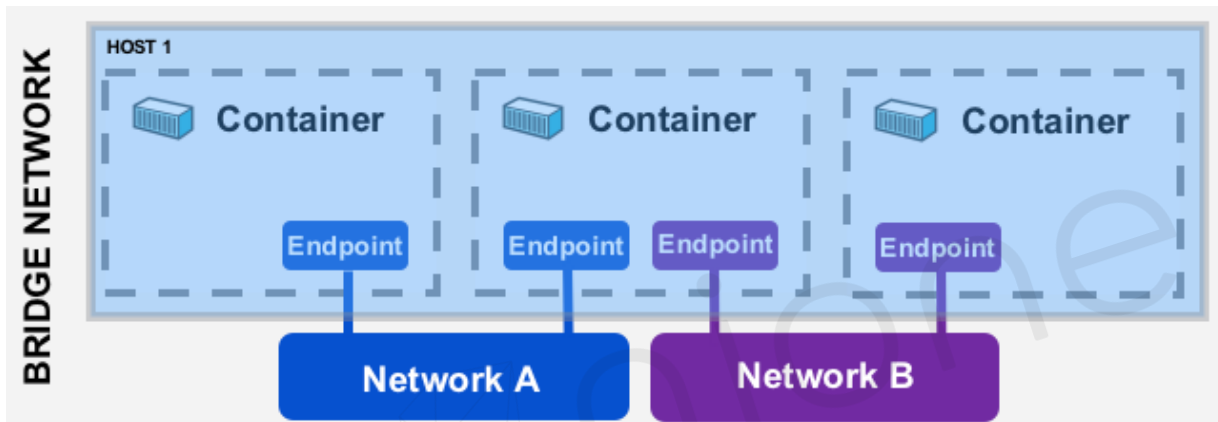
```
[root@190afa1fd4c2 ~]# docker service ls
ID                NAME        MODE         REPLICAS  IMAGE        PORTS
cm4rahh6sb7m     my-nginx    replicated    3/3        nginx:latest  *:8000->8080/tcp
[root@190afa1fd4c2 ~]# docker service ps my-nginx
```

ID	NAME	IMAGE	NODE	DESIRED STATE	STATE	CURRENT STATE	ERROR	PORTS
qwbk2zhekgrv	my-nginx.1	nginx:latest	190afa1fd4c2	Running	Running	2 minutes ago		
ki8ox24rvgoH	my-nginx.2	nginx:latest	58a5f723aafc	Running	Running	25 seconds ago		
o5heqckpmfp8	my-nginx.3	nginx:latest	e98f5674cf56	Running	Running	39 seconds ago		

Docker Swarm을 이용한 컨테이너 관리 ②

Docker Swarm Stack 활용하기

- Stack → 하나 이상의 서비스를 그룹으로 묶은 단위, 애플리케이션 전체 구성 정의
 - Docker Swarm Service는 애플리케이션 이미지를 하나 밖에 다루지 못함
 - 여러 서비스를 한꺼번에 다룰 수 있음
 - Docker Swarm Stack을 사용하여 배포된 Service 그룹은 overlay 네트워크에 속함



Docker Swarm을 이용한 컨테이너 관리 ②

Docker Swarm Stack 활용하기

- Stack 배포 (manager 실행)

```
# docker stack deploy -c /stack/stack_sample.yml my-stack
```

- 배포 된 Stack 확인

```
# docker stack services my-stack
```

- Stack에 배포 된 컨테이너 확인

```
# docker stack ps my-stack
```

- Stack삭제

```
# docker stack rm my-stack
```

Docker Swarm을 이용한 컨테이너 관리 ②

Docker Swarm Stack 활용하기

- <http://localhost:8081>



HAProxy version 2.9.7-5742051, released 2024/04/05
Statistics Report for pid 8

> General process information

pid = 8 (process #1, nbproc = 1, nbthread = 4)
uptime = 0d 0h00m42s; warnings = 2
system limits: memmax = unlimited; ulimit-n = 1048576
maxsock = 1048576; maxconn = 524270; reached = 0; maxpipes = 0
current conns = 2; current pipes = 0/0; conn rate = 0/sec; bit rate = 0.000 kbps
Running tasks: 0/23 (0 niced); idle = 100 %

Legend:

- active UP
- active UP, going down
- active DOWN, going up
- active or backup DOWN
- active or backup DOWN for maintenance (MAINT)
- active or backup SOFT STOPPED for maintenance
- backup UP
- backup UP, going down
- backup DOWN, going up
- not checked

Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

http_front																				
	Queue			Session rate			Sessions						Bytes		Denied		Errors		Warn	
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn		Resp
Frontend	0	0	-	0	0	-	2	2	524270	2	0	0	0	0	0	0	0	0	0	0

http_back																					
	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings	
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redi
app1	0	0	-	0	0	-	0	0	-	0	0	?	0	0	0	0	0	0	0	0	0
app2	0	0	-	0	0	-	0	0	-	0	0	?	0	0	0	0	0	0	0	0	0
Backend	0	0	-	0	0	-	0	0	52427	0	0	?	0	0	0	0	0	0	0	0	0

- <http://localhost:8081/haproxy?stats>

Rolling Updates & Rollback

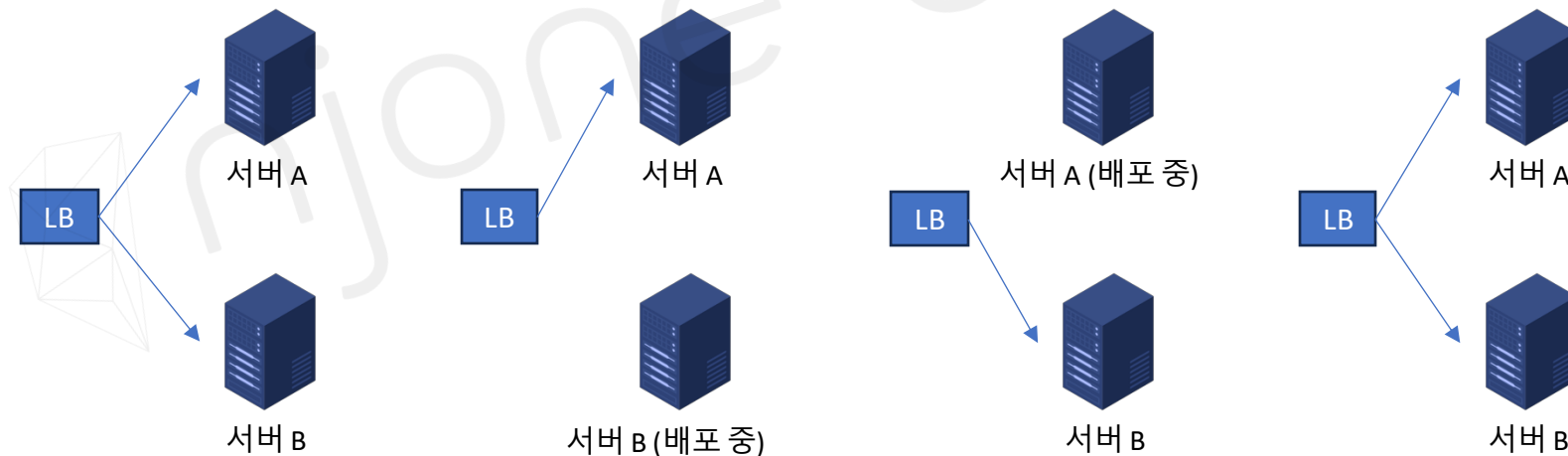
새로운 버전 배포를 위한 Rolling Updates

■ 무중단 배포

- | 서비스 장애와 배포에 있어서 부담감을 최소화 → 서비스가 중단되지 않고도 코드를 배포
- | 서비스가 중단되지 않고도 코드를 배포

■ Rolling Update

- | 서비스의 각 태스크를 한 번에 업데이트하지 않고, 지연 시간을 설정하여 태스크를 순차적으로 업데이트



Rolling Updates & Rollback

새로운 버전 배포를 위한 Rolling Updates

- Rolling Update options

- | --update-delay

- | --update-parallelism

- Service Rolling Updates (manager 실행)

- | Docker Swarm 모드는 Rolling Updates 자체 지원

```
# docker service create --name myweb --replicas 3 nginx:latest
```

```
# docker service update --image nginx:1.24 myweb
```

```
# docker service create --replicas 4 --name myweb2 --update-delay 10s --update-parallelism 2 nginx:latest
```

```
# docker service update --image nginx:1.24 myweb2
```

Rolling Updates & Rollback

새로운 버전 배포를 위한 Rolling Updates

- Service Rolling Updates 확인 (Container가 실행 된 Node에서 확인)

| docker inspect [container_id] | grep Created

```
[root@41b1cd4a5143 ~]# docker inspect 8628b364cec7 | grep Created
"Created": "2024-05-08T23:20:47.567711758Z",
[root@41b1cd4a5143 ~]# docker inspect 1966b027a637 | grep Created
"Created": "2024-05-08T23:20:47.548085633Z",
```

```
[root@92d25d051266 ~]# docker inspect 0c62b51eed41 | grep Created
"Created": "2024-05-08T23:21:00.489221651Z",
```

```
[root@479b4f5e69e4 ~]# docker inspect dbfaf1d1c5a9 | grep Created
"Created": "2024-05-08T23:21:00.649897902Z",
```

```
# docker service create --replicas 4 --name myweb2 --update-delay 10s --update-parallelism 2 nginx:latest
```

Rolling Updates & Rollback

이전 버전으로 돌아가기 위한 Rollback

- Docker Swarm Rollback (manager 실행)

```
# docker service create --name redis \  
  --replicas 4 --rollback-delay 10s --rollback-parallelism 1 \  
  --rollback-failure-action pause redis:7.0.3
```

```
# docker service inspect redis --pretty
```

```
# docker service update --image redis:7.0.4 redis
```

```
# docker service update --rollback redis
```

Rolling Updates & Rollback

도커 이미지 관리

- 작업 중인 컨테이너를 이미지로 저장

| docker commit

```
$ docker commit <CONTAINER_NAME> <IMAGE_NAME:TAG_NAME>
```

- 백업 / 복원

| docker save / load

```
$ docker save <OPTIONS> <TAR_FILENAME> <IMAGE_NAME:TAG_NAME>
```

```
$ docker load -i <TAR_FILENAME>
```